

Abstract

Shared-memory data structures must guarantee correctness under concurrent access from multiple processes. This report describes the design and execution of a test suite that validates the correctness and robustness of the ``pstree`` library, a POSIX-based shared-memory binary search tree implementation when subjected to concurrent insert, delete, and get operations from independent processes. We examine startup synchronization issues, demonstrate how small sleep delays avoid spurious failures, detail the library's mutex-based protection, and present observed outcomes. The results confirm that, once the shared memory region is properly initialized, all tree operations execute safely without race conditions or busy-waiting, fully satisfying the project's concurrency requirements.

1 Introduction

Concurrent data structures in shared memory must address two main challenges: (1) ensuring that concurrent operations on the data structure do not corrupt its internal state, and (2) guaranteeing that the structure can be safely created and opened by clients in any order without spurious failures. The ``pstree`` library provides a shared-memory binary search tree (BST) abstraction for inter-process communication, offering functions to create, open, close, insert, delete, and retrieve key-value pairs from a global tree stored in an ``mmap``-backed region. Synchronization is achieved entirely via a ``PTHREAD_PROCESS_SHARED`` mutex stored within the shared segment; no spin loops or busy-waiting are employed.

This report documents a 4 tests harness designed to exercise the ``pstree`` implementation under real concurrency. We describe the test applications, the execution procedure, and the rationale behind adding small sleep intervals to avoid startup races. Finally, we present the observed results and discuss their implications for correctness and performance.

2 Experimental Setup and Environment

All experiments were conducted on an Ubuntu 22.04 LTS (64-bit) virtual machine with 8 GB RAM and a 8 processor virtual CPU. The library and test programs were compiled with GCC 11.2 using the following commands:

```
bash
```

```
ar rcs libpstree.a pstree.o
```

```
gcc -g -Wall -o app1-add app1-add.o -L. -lpstree
```

```
gcc -g -Wall -o app1-get app1-get.o -L. -lpstree
```

```
gcc -g -Wall -o app2-insert app2-insert.o -L. -lpstree
```

```
gcc -g -Wall -o app2-get app2-get.o -L. -lpstree
```

```
gcc -g -Wall -o app3-insert-delete app3-insert-delete.o -L. -lpstree
```

```
gcc -g -Wall -o app3-get app3-get.o -L. -lpstree
```

```
gcc -g -Wall -o app4-mixed app4-mixed.o -L. -lpstree
```

...

Each program links against ``libpstree.a`` and operates on the shared-memory object named ``/tree1``.

3 Test Applications

Seven small standalone C programs exercise different aspects of tree manipulation, this implement in total 4 tests:

Program	Main Job
app1-add	Create <code>`/tree1`</code> and insert keys 1–10
app1-get	Report node count; read keys 1–10
app2-insert	Insert keys 100–109
app2-get	Read keys 100–109 after insertion
app3-insert-delete	Insert 200–203; delete 201; insert 203
app3-get	Verify presence of 200, 202, 203; confirm 201 is absent
app4-mixed	Insert, get, and delete keys 300–303; finally destroy the tree

Except for ``app1-add`` (the sole creator) and ``app4-mixed`` (which calls ``pst_destroy``), all programs invoke only ``pst_open`` on ``/tree1``, ensuring they operate on the same shared-memory segment.

4 Execution Procedure

To achieve true concurrency, we launch each program in the background from a single shell. The basic invocation is:

```
bash
Terminal: concurrent launch without startup delay
./app1-add &
./app1-get &
./app2-insert &
./app2-get &
./app3-insert-delete &
./app3-get &
./app4-mixed &
wait
```

This causes all seven processes to overlap in time, leading to contention on the shared mutex and exercising the library's synchronization logic. However, without any coordination, some clients may attempt to open the tree before it is fully created and initialized, resulting in spurious "Error opening tree" messages or invalid node counts.

5 Startup Synchronization and Sleep Usage

5.1 Observed Startup Race

During initial trials, reader and inserter programs occasionally failed because they called ``pst_open("/tree1")`` before ``app1-add`` had completed the ``shm_open`` + ``ftruncate`` + ``mmap`` sequence and initialized the shared mutex. In such cases, ``pst_open`` returned ``PST_ERROR`` and set ``pst_errorcode`` to ``ERR_shm_open`` or mapped a zeroed header, leading to incorrect node counts such as -1 which is the result of `PST_ERROR` value being returned.

5.2 Avoiding Failure with Sleep Delays

To mitigate this ordering race without introducing complex rendez-vous primitives, we inserted short ``sleep 2`` intervals between startup phases:

```
bash
Serialized startup with overlap-preserving sleeps
./app1-add
sleep 2
./app1-get & ./app2-insert &
sleep 2
./app2-get &
./app3-insert-delete &
sleep 2
./app3-get & ./app4-mixed &
wait
```

These sleeps guarantee that:

1. ``app1-add`` finishes creating and initializing the tree before any ``pst_open`` by other processes.
2. ``app2-insert`` completes (or is well underway) before ``app2-get`` begins.
3. ``app3-insert-delete`` has deleted key 201 before ``app3-get`` queries it.

Despite the sleeps, the processes still overlap and contend on the shared mutex during their critical sections. The delays affect only startup ordering, not the core concurrency testing.

5.3 In Production: Robust Startup Protocols

In this project, we were able to handle the startup order by just putting a small sleep 2 after running app1-add.

This gave enough time for the tree to be created before other programs tried to open it. It worked fine for our demo and kept everything simple. But if this was a real production system, just relying on sleep wouldn't be enough. There's no guarantee that the memory would always be ready exactly after 2 seconds. Maybe the system is slower, maybe it's faster, we can't know with certainty. Blindly sleeping can waste resources and time and even after that it can still fail if the memory isn't ready yet.

That's why in real-world software, people use better ways to handle this kind of timing.

Here's what could be a possible way to do it instead:

- **Retry Loop in pst_open:**

If opening the shared memory fails (for example, the file /tree1 doesn't exist yet),

you just wait a tiny bit (like 100ms) and then try again.

Keep retrying a few times before giving up.

This way you don't rely on a fixed sleep, you react based on the real situation.

- **Named Semaphores or Events:**

The program that creates the tree (app1-add) could "signal" when it's fully ready (using a semaphore or event). Other programs would wait for this signal before starting. This is very clean and reliable, and nobody wastes time sleeping randomly.

- **Launcher Script with Dependencies:**

Another simple option is to write a small script that first runs app1-add, waits for it to finish, and only then starts the other apps. This way you are manually controlling the order without needing the apps to figure it out themselves.

In short, for our project, a fixed sleep was good enough. It kept things simple and let us focus on testing concurrency without overcomplicating startup. But in a real production setup, you'd want something more reliable than a fixed delay. Handling startup properly would make the system faster, cleaner, and safer under all conditions.

6 Library-Side Synchronization

The core of ``pstree.c`` relies on a single ``pthread_mutex_t lock`` stored in the shared memory header:

c

```
pthread_mutex_lock(&header->lock);
```

```
    // walk or modify the tree structure
```

```
pthread_mutex_unlock(&header->lock);
```

Mutual Exclusion: All modifying (``pst_insert``, ``pst_delete``) and reading (``pst_get``) operations acquire the same mutex, ensuring that only one process at a time traverses or mutates the tree, thus preventing data races.

No Busy-Waiting: The library does not use spin loops; blocked threads sleep inside the kernel until the mutex becomes available.

Process-SharedAttribute: ``pthread_mutexattr_setpshared(&mutex_attr, PTHREAD_PROCESS_SHARED)`` ensures the mutex works across distinct address spaces.

Because ``pst_open`` either succeeds (mapping a fully initialized header) or fails early without touching the mutex, there is no risk of deadlock or undefined behavior on startup failures.

7 Results and Discussion

After introducing startup sleeps, all six processes executed successfully in every trial. Representative outputs are summarized below.

7.1 With Sleep Delays

```
app1-get    → there are 10 nodes in the tree; data received; datasize=...
app2-insert → Inserted key 100 ... Inserted key 109
app2-get    → Key 100 found, data=bulk100 ... Key 109 found, data=bulk109
app3-insert-del → inserted 200–203, deleted 201
app3-get    → Key 200 found, Key 201 not found, ... Key 203 found
app4-mixed  → Inserted 300; Found 300; Deleted 301; Found 303; pst_destroy
```

7.2 Without Sleep Delays

```
app1-add : created tree and inserted 1–10
app1-get : "there are -1 nodes in the tree" or "could not find key"
app2-insert : may proceed
app2-get  : "Key 100 not found" ... spuriously
app3-get, app4-mixed : unreliable reads or errors
```

No data corruption, duplicate keys, or unexpected crashes were ever observed once the tree was opened successfully. The mutex effectively serialized access to the tree, and condition variables—though initialized—were not required for this test scenario.

8 Conclusion and Future Work

Our concurrent test suite confirms that the ``pstree`` library correctly handles simultaneous inserts, deletes, and gets from multiple processes when the shared memory region is properly initialized. The only startup issues were due to ordering between the creator and consumers; these were safely mitigated with brief sleeps. All subsequent operations relied on a single POSIX mutex to ensure atomicity and consistency, with no busy-waiting. No true race condition was created, the only issue faced without using sleep was an attempt to access memory without proper initialisation being completed which was also handled easily by library in-built errors.